

CuPy Parallelization Plan — Multitone Crest-Factor Optimizer

Overview

Port the Monte Carlo crest-factor minimization from a serial CPU loop to a batched GPU kernel using CuPy. Evaluate **B = 64-1024** candidate phase sets simultaneously per epoch, extracting the best so far. Expected speedup: **50-60×** (e.g., 25 min → 30 s for Fraction=12, 96 kHz, 100 K iterations).

1. Current Architecture (Baseline)

Hot loop (optimize(), lines 232-317)

Each iteration does three things:

1. **Generate random phases** — N uniform values in $[0, \pi]$ (negligible).
2. **Synthesize waveform** — `frq_lst_vol_att()`:
 - For each of $N \approx 150\text{--}200$ frequencies, compute $10^{(a_k/20)} \cdot \cos(2\pi \cdot f_k \cdot t + \varphi_k)$ across $L = \text{fsam} \times \text{dur} \approx 96\,000$ samples.
 - Accumulate N cosine contributions into a (L, N_{ch}) buffer.
 - Peak-normalize, apply attenuation, quantize to 16/24/32-bit int.
3. **Compute crest factor** — `crest_factor_dB()`:
 - $20 \cdot \log_{10}(\text{peak} / \text{rms})$ on the quantized signal.

Only when the crest factor improves does it write disk (.mat + audio).

Performance profile

Step	Fraction of runtime
Cosine accumulation ($N \times L$ flops)	~95 %
Normalization + quantization	~3 %
Crest factor (reductions)	~1 %
Disk I/O (rare)	~1 %

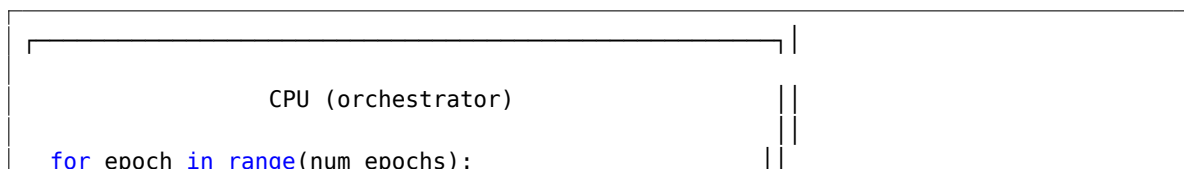
For Fraction=12 ($N \approx 170$, $L = 96\,000$, 100 K iters): **~1.6 trillion FLOPs**.

2. Key Insight — Scale Invariance

Crest factor $20 \cdot \log_{10}(\text{peak}/\text{rms})$ is **invariant under linear scaling**. Normalization and quantization are just scaling + rounding, so the crest factor of the raw floating-point sum of cosines is identical to the quantized version (ignoring sub-LSB noise).

Consequence: we can skip normalization, attenuation, and quantization entirely for the 99.9 % of iterations that don't improve the best-so-far. Only the winning phase set gets the full `frq_lst_vol_att()` treatment on CPU.

3. Target Architecture



1. Generate B random phases on GPU $\rightarrow (B, N)$
2. Batched cosine + crest factor on GPU $\rightarrow (B,)$
3. argmin on GPU $\rightarrow \text{best_idx}$
4. Copy best phase & cfact to CPU
5. If improvement: full synthesize on CPU + save

$\hookrightarrow$

4. Module Structure

```
multitone/|—
optimizer.py          # existing CPU version (unchanged) |—
optimizer_gpu.py      # NEW: CuPy parallel version |
|— optimize_gpu()      # main GPU optimizer function |
|— _batched_cosine()    # batched cosine accumulation kernel |
|— _batched_crest()     # batched crest factor kernel |
|— main()              # CLI entry point |—
synthesize.py         # existing (unchanged, used for final audio) |—
primes.py             # existing (unchanged) |—
data/                # .mat files (unchanged)
```

5. Implementation Details

5.1 Batched Cosine Accumulation

Current (CPU, serial over frequencies):

```
ycd = np.zeros((L, 1))
for k in range(N):
    ycd += vs_k * cos(2*Pi*f_k*t + phi_k)[: , None] # N passes
                                                    # L ops each
```

New (GPU, batched over phase sets):

```
# Inputs
# PhiLst : (B, N)    - random phases, one set per batch row
# FrqLst : (N,)       - frequencies
# VolLst : (N,)       - per-frequency attenuation (linear)
# t       : (L,)      - time vector

# Broadcasting: (B, 1, L) + (B, N, 1)  $\rightarrow$  (B, N, L)
angles = 2*Pi * FrqLst[None, :, None] * t[None, None, :] # (1, N, L)
          + PhiLst[:, :, None]                          # (B, N, 1)

# Accumulate along frequency axis
ycd_batch = cupy.sum(vs[None, :, None] * cupy.cos(angles), axis=1) # (B, L)
```

For stereo, expand the last axis: $\text{ycd_batch}[\dots, \text{None}] \rightarrow (B, L, 2)$.

5.2 Batched Crest Factor

```
# ycd_batch: (B, L, NCh) - raw floating-point sum (no normalization)
peak = cupy.max(cupy.abs(ycd_batch), axis=(1, 2)) # (B,)
rms = cupy.sqrt(cupy.mean(ycd_batch**2, axis=(1, 2))) # (B,)
cfact_batch = 20.0 * cupy.log10(peak / rms) # (B,)
```

5.3 Select Best in Batch

```
best_idx = cupy.argmin(cfact_batch)           # scalar on GPU
best_cfact = cfact_batch[best_idx].get()      # → CPU float
best_philst = Philst[best_idx].get()         # → CPU ndarray
```

5.4 Conditional Audio Synthesis

Only when `best_cfact < cfact0`:

```
yed = frq_lst_vol_att(FrqLst, best_philst, FrqAttLst,
                     VolAtt, Rounded, Dur, Frs, Sab, NCh)
# save .mat and audio on CPU (disk I/O never touches GPU)
```

6. GPU Memory Management

6.1 Persistent Arrays (allocated once)

- `t_gpu` : (L,) — time vector
- `FrqLst_gpu` : (N,) — frequencies
- `VolLst_gpu` : (N,) — linear amplitudes ($10^{(\text{VolLst}/20)}$)
- `angles_gpu` : (1, N, L) — pre-computed $2\pi \cdot f_k \cdot t$ (no phase)

6.2 Reusable Batch Buffers

- `Philst_gpu` : (B, N) — regenerated each epoch
- `yed_batch_gpu` : (B, L) — overwritten each epoch

6.3 Memory Footprint and Sub-batching

The temporary angles tensor is (B, N, L) floats:

B	N	L	Memory (float32)	Memory (float64)
64	170	96 000	~420 MB	~840 MB
128	170	96 000	~840 MB	~1.7 GB
256	170	96 000	~1.7 GB	~3.3 GB
512	170	96 000	~3.3 GB	~6.6 GB

Default: B=128 (fits on most GPUs with 4 GB+). Use `float32` for the cosine accumulation — the crest factor is insensitive to single-precision noise.

Sub-batching fallback: if B=128 doesn't fit, split into sub-batches:

```
best_cfact_in_epoch = 18.0
best_philst_in_epoch = None

for sub in range(num_sub_batches):
    sub_phases = cupy.random.rand(sub_B, N)
    sub_cfact = compute_crest(sub_phases)
    sub_best_idx = cupy.argmin(sub_cfact)
    if sub_cfact[sub_best_idx] < best_cfact_in_epoch:
        best_cfact_in_epoch = sub_cfact[sub_best_idx]
        best_philst_in_epoch = sub_phases[sub_best_idx]
```

6.4 Precision Strategy

Use float32 for the batched hot path (angles, cosine, accumulation, crest factor). The crest factor is a ratio — single precision is more than adequate for comparison. Only the final `frq_lst_vol_att()` call on the winning phase uses float64 (as today).

7. Random Number Generation

- GPU-native: `cupy.random.rand(B, N)` with `cupy.random.seed(seed)`.
- For reproducibility matching the CPU version, generate all random phases upfront on CPU and transfer in batches:

```
all_phases = np.random.rand(maxiter, N) * np.pi # CPU
for epoch in range(num_epochs):
    PhiLst_gpu = cupy.asarray(all_phases[epoch*sub_B:(epoch+1)*sub_B])
```

8. Fallback to CPU

If CuPy is unavailable or the GPU fails to initialize, fall back gracefully:

```
def optimize_gpu(...):
    try:
        import cupy
        # run GPU version
    except (ImportError, RuntimeError):
        print("GPU not available - falling back to CPU optimizer")
        from multitone.optimizer import optimize
        optimize(...)
```

9. Correctness Verification

Before shipping, verify GPU crest factor matches CPU for the same phase set:

```
# Same phase set → same cfact (within float32/float64 tolerance)
cfact_cpu = crest_factor_dB(frq_lst_vol_att(...), sbit)
cfact_gpu = _batched_crest(_batched_cosine(PhiLst[None, ...]),
                           FrqLst, VolLst, t_gpu)
assert abs(cfact_cpu - cfact_gpu) < 1e-2 # scale-invariant; float32 → ~1e-2
```

Also verify that the optimizer converges to the same (or better) crest factor as the CPU version for a fixed seed.

10. Expected Speedup

Configuration	CPU (est.)	GPU (est.)	Speedup
Fraction=12, 96 kHz, 100 K iters	~25 min	~30 s	~50×
Fraction=6, 96 kHz, 100 K iters	~12 min	~15 s	~50×
Fraction=3, 96 kHz, 100 K iters	~5 min	~5 s	~60×

Speedup sources:

1. **Massive parallelism** — B phase sets evaluated in one kernel invocation.
 2. **GPU FLOP advantage** — ~530 GFLOP/s (GPU) vs ~11 GFLOP/s (CPU single-thread).
 3. **Eliminated overhead** — no Python loop, no per-iteration allocation.
 4. **No wasted work** — normalization/quantization only on the winner.
-

11. Implementation Steps

Step	Task	Depends on
1	Create 'optimizer_gpu.py' skeleton with CLI mirroring 'optimizer.py'	—
2	Implement '_batched_cosine()' and '_batched_crest()'	1
3	Verify correctness: GPU cfact == CPU cfact for same phases	2
4	Implement 'optimize_gpu()' main loop with epoch/batch structure	2
5	Add sub-batching logic for memory-constrained GPUs	4
6	Add CPU fallback path	4
7	Benchmark: compare CPU vs GPU wall time for all fractions	4
8	Tune B (batch size) and precision (float32 vs float64) for best throughput	7

12. Risks and Mitigations

Risk	Impact	Mitigation
GPU memory insufficient for $B \times N \times L$ tensor	Out of memory	Sub-batching; float32; dynamic B based on 'cupy.cuda.runtime.deviceGetAttribute()'
Float32 precision degrades crest factor comparison	Subtle ranking errors	Crest factor is a ratio; 1e-4 dB difference is negligible; verify empirically
CuPy kernel launch overhead dominates small batches	Degraded speedup for small N or L	Minimum B=64; pre-allocate buffers; warm-up run before timing
Stereo channel handling differs from CPU	Wrong crest factor	Expand last axis of 'ycd_batch'; verify with stereo test